Algoritmos e Estrutura de Dados

Teoria e Implementações

J. P. S. de Sousa

Rio de Janeiro — 2026

Sumário

I Estruturas de Dados Elementares **1**

1 Estruturas Sequenciais **2**

1.1 Vetores 3

1.1.1 Vetores Não Ordenados 3

1.1.2 Vetores Ordenados 5

1.2 Listas Encadeadas 6

1.2.1 Lista Simplesmente Encadeada 6

1.2.2 Lista Duplamente Encadeada 7

1.2.3 Lista Circular Duplamente Encadeada 8

1.3 Pilha 9

1.4 Filas 10

Parte I

Estruturas de Dados Elementares

Capítulo 1

Estruturas Sequenciais

Na matemática, conjuntos são coleções imutáveis de objetos distintos. Na ciência da computação, conferimos a nomenclatura de **conjuntos** a qualquer coleção finita abstrata de objetos. A imutabilidade, nesse caso, não é um requisito; conjuntos computacionais podem crescer ou diminuir de tamanho ao longo das operações de um algoritmo que o manipula. Tais conjuntos que podem ter seu tamanho alterado chamamos de **dinâmicos**. Os conjuntos dinâmicos que também permitem o teste de pertinência de um membro dado são chamados de **dicionários**.

Os elementos de um conjunto de dados são, geralmente, um registro, cujos campos podem ser de tipos elementares, ponteiros, ou compostos de outros registros. Os elementos num conjunto não necessariamente precisam ser todos distintos entre si, a exemplo da coleção de idades de um grupo de pessoas; nada impede que haja duas com a mesma idade. Em alguns casos, o membros de um conjunto possuem uma **chave** que os identifica, e, caso as chaves sejam todas distintas, o conjunto pode ser considerado como uma coleção de chaves.

As operações de um conjunto podem ser de dois tipos: consulta ou modificação. A primeira apenas resgata um valor, membro ou estatística do conjutno. A segundo altera o estado do conjunto, como sua ordenação, inserindo, removendo ou modificando membros, entre outras. Em geral, quase todo conjunto aceita as seguintes operações:

Insere(S, x**)** Operação que toma um conjunto S e insere o membro apontado por x. Presume-se que tanto S quanto x já foram inicializados. Dependendo da aplicação, a operação pode falhar caso um membro x com a mesma chave já exista em S.

Remove(S, x**)** Remove do conjunto S o membro apontado por x.

Busca(S, k**)** Retorna um ponteiro para um membro x em S cuja chave seja igual a k, ou **NIL** caso nenhum elemento contenha tal chave.

Outras operações básicas que acompanham conjuntos dinâmicos são

Máximo(S) Retorna o membro x de S cujo valor da chave é máximo no conjunto

Mínimo(S) Retorna o membro x de S cujo valor da chave é mínimo no conjunto

Predecesor(S, x) Num conjunto ordenado, retorna o membro $w \in S$ que precede x, ou **NIL** quando x é o elemento mínimo.

Sucessor(S, x) Num conjunto ordenado, retorna o membro $y \in S$ que sucede x, ou **NIL** quando x é o elemento máximo.

1.1 Vetores

Um vetor é um conjunto de objetos de mesmo tipo[1] que são armazenados contiguamente na memória. Seus elementos podem ser acessados diretamente pelos seus índices. Se o nome do vetor é S, então acessamos seu i-ésimo membro como $S[i]$. O nome do vetor é um ponteiro para o seu primeiro elemento, e, ao somá-lo com um valor inteiro não negativo i, obtemos um ponteiro para o elemento de índice i do vetor, portanto,

$$S[i] = *(S + i)$$

Na sua formulação mais simples, vetores possuem dois atributos principais: tamanho – o número máximo de elementos que ele suporta – e o número efetivo de membros. Chamamos de estáticos os vetores que possuem tamanho previamente determinado e fixo, enquanto são denominados dinâmicos aqueles cujo tamanho varia conforme a necessidade da aplicação. Nas implementações expostas, presume-se que os vetores são dinâmicos.

1.1.1 Vetores Não Ordenados

A busca em vetores não ordenados é realizada percorrendo o vetor sequencialmente até achar o elemento com a chave desejada. No pior caso, o elemento ou está na última posição, ou não existe no vetor. Com isso, a sua complexidade é $O(n)$. Para vetores, é conveniente retornar o índice do elemento sendo buscado, e, caso ele não exista, retornamos -1 no lugar.

Função Busca-Vetor(S, k)

```
1 i ← 0;
2 enquanto i < S.num_elem e S[i].chave ≠ k faça
3 |   i ← i + 1;
4 fim
5 se i = S.num_elem então
6 |   retorna −1;
7 fim
8 retorna i;
```

[1] Isso porque dois dados do mesmo tipo possuem o mesmo número de bytes na memória.

A inserção num vetor pode receber como argumentos um ponteiro para o objeto a ser inserido e o índice da posição em que ele será colocado. Como pode ser necessário mover os membros já presentes para inserir um novo, ou realocar o vetor quando esse já está cheio, a inserção tem complexidade $O(n)$.

Função Insere-Vetor(S, x, i)

```
1  se i < 0 ou i > S.num_elem então
2  |  erro;
3  fim
4  se S.num_elem = S.len então
5  |  S ← Realoca-Vetor(S);
6  fim
7  j ← S.num_elem;
8  enquanto j ≠ i faça
9  |  S[j] ← S[j − 1];
10 |  j ← j − 1;
11 fim
12 S[i] ← ∗x;
13 S.num_elem ← S.num_elem + 1;
```

Contudo, se o vetor for um conjunto não ordenado, então podemos sempre inserir o novo elemento logo após o último membro, de forma a não executar o laço que move elementos a partir do índice i uma posição adiante. Com isso, a inserção assume complexidade $O(1)$ a menos de uma realocação com custo $O(n)$.

Função Realoca-Vetor(S)

```
1  T ← aloca(S.len ∗ 2);
2  para i ← 0 até S.num_elem − 1 faça
3  |  T[i] ← S[i];
4  fim
5  T.num_elem ← S.num_elem;
6  T.len ← S.len · 2;
7  retorna T;
```

Em vetores não ordenados, a remoção pode ser realizada simplesmente copiando o último elemento para a posição daquele que está sendo retirado. Desse modo, a remoção é executada em tempo constante.

Função Remove-Vetor(S, i)

```
1 se S.num_elem = 0 então
2 |   erro "underflow";
3 fim
4 se i < 0 ou i ≥ S.num_elem então
5 |   erro;
6 fim
7 S[i] ← S[S.num_elem − 1];
8 S.num_elem ← S.num_elem − 1;
```

1.1.2 Vetores Ordenados

Em vetores ordenados, geralmente em ordem crescente, a busca pode ser feita em tempo sublinear $O(\log n)$ se usarmos o algoritmo de busca binária, que será apresentado num capítulo mais adiante. Com ela, em vez de retornar -1 quando a chave não é encontrada, retornamos o índice que o elemento com a chave buscada ocuparia caso estivesse no vetor. Na inserção, realiza-se a busca pela chave do elemento a ser inserido no vetor para obter o índice no qual ele deve ser posicionado. Depois, passa-se esse índice para o mesmo procedimento de inserção apresentado para vetores não ordenados.

Remoções em vetores ordenados requerem que todo membro que sucedia aquele que está sendo retirado seja movido para uma posição atrás. Por consequência, a complexidade da operação é $O(n)$.

Função Remove-Vetor-Ordenado(S, i)

```
1  se S.num_elem = 0 então
2  |   erro "underflow";
3  fim
4  se i < 0 ou i ≥ S.num_elem então
5  |   erro;
6  fim
7  j ← i;
8  enquanto j < S.num_elem − 1 faça
9  |   S[j] ← S[j + 1];
10 |   j ← j + 1;
11 fim
12 S.num_elem ← S.num_elem − 1;
```

Para as operações de consulta de máximo e mínimo, basta retornar o ponteiro para o último e o primeiro elemento no vetor respectivamente, sendo, pois, triviais por meio da indexação. O mesmo vale para o predecessor e sucessor do elemento de índice i, que consistem nos elementos de índices $i - 1$ e $i + 1$ respectivamente quando esses são válidos, senão, **NIL** deve ser retornado.

1.2 Listas Encadeadas

Lista encadeada é uma implementação de conjunto dinâmico cujos membros, denominados nós, nas suas formas mais simples, detêm um campo de chave e um ponteiro `proximo` que aponta para o próximo membro no conjunto. Numa implementação típica, a lista possui dois ponteiros especiais: `head` e `tail`. O `head` aponta para o primeiro elemento, enquanto o segundo para o último. No entanto, para simplificar condições de contorno nas operações de inserção e remoção, principalmente no início da lista, adiciona-se um nó chamado de `sentinela`, que não possui chave válida, é apontado pelo `head` e cujo `proximo` aponta para o primeiro membro efetivo da lista.

As listas encadeadas possuem diversas variações: listas simplesmente encadeadas, duplamente encadeadas e circulares. Também podemos contar com variações que incluem elementos de mais de um tipo. De forma geral, assim como em vetores não ordenados, é possível realizar inserções e remoções de um dado membro em tempo constante caso a ordenação não seja requisitada, e buscas em tempo linear.

As listas encadeadas possuem a vantagem de serem mais flexíveis e permitirem inserções sem necessariamente conhecer o total de elementos a serem comportados, diferentemente dos vetores, que precisam ser realocados à medida que o número de membros cresce. No entanto, os vetores fazem melhor aproveitamento da cache devido ao princípio de localidade espacial. No caso de existência de ordenação, a vantagem das listas encadeadas diminui mais, pois uma busca num vetor ordenado pode ser executada em tempo logarítmico, enquanto nas listas continua sendo realizado em tempo linear.

1.2.1 Lista Simplesmente Encadeada

Consiste na implementação mais simples para uma lista encadeada. A lista mantém três atributos: `head`, `tail` e o número de elementos. Uma busca nessa lista consiste numa pesquisa sequencial: percorremos cada elemento, seguindo os ponteiros de `proximo` até obter a chave desejada. É importante frisar que o que é retornado não é um ponteiro para o membro cuja chave é aquela sendo pesquisada, mas sim um ponteiro para o membro anterior. Isso porque um nó não conhece aquele que o antecede, apenas o que lhe sucede.

Função Busca-Lista(L, k)

```
1 w ← L.head;
2 enquanto w.proximo ≠ NIL e w.proximo.chave ≠ k faça
3 |   w ← w.proximo;
4 fim
5 se w.proximo = NIL então
6 |   retorna NIL;
7 fim
8 retorna w;
```

A inserção é feita tomando-se como argumento o nó `prev` que antecederá o novo membro do conjunto. Neste momento, é feito o ajuste dos ponteiros de `proximo`, a

fim de que o `proximo` de `prev` aponte para o novo membro, e o `proximo` desse último aponte para o antigo sucessor de `prev`. Se `prev` era o `tail` da lista, então o novo nó será o novo `tail`, o que pode ser feito verificando-se se o `proximo` do novo é **NIL**.

Função Insere-Lista(L, prev, x)

```
1 x.proximo ← prev.proximo;
2 prev.proximo ← x;
3 se x.proximo = NIL então
4 |   L.tail ← x;
5 fim
6 L.num_elem ← L.num_elem + 1;
```

A remoção toma como argumento o nó `prev` que precede aquele que será removido. Nessa operação, o `proximo` de `prev` irá apontar para o `proximo` do nó retirado. Se o removido era o `tail`, então `prev` passa a ocupar essa função.

Função Remove-Lista(L, prev)

```
1 se prev.proximo = NIL então
2 |   retorna;
3 fim
4 x ← prev.proximo;
5 prev.proximo ← x.proximo;
6 se prev.proximo = NIL então
7 |   L.tail ← prev;
8 fim
9 L.num_elem ← L.num_elem − 1;
```

1.2.2 Lista Duplamente Encadeada

Essa variante funciona de maneira análoga à encadeada simples. O adicional é que os membros agora possuem um ponteiro de `anterior`, que aponta para o nó antecessor. Nesse caso, a busca pode retornar um ponteiro para o próprio elemento que possui a chave pesquisada.

Função Busca-Lista-D(L, k)

```
1 w ← L.head.proximo;
2 enquanto w ≠ NIL e w.chave ≠ k faça
3 |   w ← w.proximo;
4 fim
5 se w = NIL então
6 |   retorna NIL;
7 fim
8 retorna w;
```

Função Insere-Lista-D(L, prev, x)

```
1 x.proximo ← prev.proximo;
2 prev.proximo ← x;
3 x.anterior ← prev;
4 se x.proximo = NIL então
5 |   L.tail ← x;
6 senão
7 |   x.proximo.anterior ← x;
8 fim
9 L.num_elem ← L.num_elem + 1;
```

Função Remove-Lista-D(L, x)

```
1 se x = L.head então
2 |   retorna;
3 fim
4 prev ← x.anterior;
5 prev.proximo ← x.proximo;
6 se prev.proximo = NIL então
7 |   L.tail ← prev;
8 senão
9 |   x.proximo.anterior ← prev;
10 fim
11 L.num_elem ← L.num_elem − 1;
```

1.2.3 Lista Circular Duplamente Encadeada

A lista encadeada circular é aquela em que o último nó aponta para o primeiro em vez de **NIL**, ou seja, o `head` da lista equivale a L.tail.proximo. Ao adicionar o ponteiro `anterior` a cada membro, obtemos, então, a lista circular duplamente encadeada, que combina todos os elementos vistos até então sobre listas. As operações de busca, inserção e remoção são análogas àquelas vistas para listas duplamente encadeadas.

Iremos expor a implementação dessa variação usando o nó sentinela. Com a lista vazia, os ponteiros `proximo` e `anterior` da sentinela apontam para ela mesma. À medida que as inserções ocorrem, esses ponteiros são ajustados para refletir a estrutura circular.

A busca continua retornando um ponteiro para o nó que contém a chave desejada. O que muda é a condição de parada quando a chave não existe. Em vez de encerrar quando atingir **NIL**, o critério de parada passa a ser revisitar a sentinela.

A estrutura circular elimina a condição de contorno de quando inserimos ou eliminamos no final da lista. Desse modo, as inserções e remoções funcionam da mesma forma independentemente de onde as realizamos na lista. A única operação que deve ser coibida é a remoção da própria sentinela.

Função Busca-Lista-DC(L, k)

```
1 w ← L.head.proximo;
2 enquanto w ≠ L.head e w.chave ≠ k faça
3 |   w ← w.proximo;
4 fim
5 se w = L.head então
6 |   retorna NIL;
7 fim
8 retorna w;
```

Função Insere-Lista-DC(L, prev, x)

```
1 x.proximo ← prev.proximo ;
2 prev.proximo ← x;
3 x.anterior ← prev;
4 x.proximo.anterior ← x;
5 L.num_elem ← L.num_elem + 1;
```

Função Remove-Lista-DC(L, x)

```
1 se x = L.head então
2 |   retorna;
3 fim
4 prev ← x.anterior;
5 prev.proximo ← x.proximo;
6 x.proximo.anterior ← prev;
7 L.num_elem ← L.num_elem − 1;
```

1.3 Pilha

Uma pilha é uma estrutura de dados abstrata que define uma política na ordem das inserções e remoções. Essa política é denominada **LIFO**: o último elemento a entrar é sempre o primeiro a ser removido. Uma analogia é uma pilha de pratos num restaurante; normalmente, os pratos colocados acima, que correspondem aos últimos a terem sido adicionados, são os primeiros a serem removidos. Ao manipular uma pilha, a operação de inserção é denominada *push*, enquanto a de retirada é denominada *pop*.

Há pelo menos duas maneiras de implementar pilhas: com vetores ou com listas encadeadas simples. Independentemente da estrutura subjacente escolhida, é importante garantir que tanto a adição quanto a remoção possuam custo constante em tempo.

Na implementação com vetor, basta adicionar o novo elemento ao final. Dependendo do requisito da aplicação, realocações podem ou não ser realizadas. Caso não o sejam, um erro de *stack overflow* deve ser levantado. Já para a operação de *pop*, basta retirar o elemento na última posição. Usualmente, seu conteúdo é retornado para impedir que ele seja perdido. Dessa forma, cabe a quem chama o procedimento decidir se irá ou

não salvar o objeto devolvido.

```
1  função push(S, x)
2      se S.num_elem = S.len então
3          erro "stack overflow";
4      fim
5      Insere-Vetor(S, x, S.num_elem);
6  fim
7  função pop(S)
8      se S.num_elem = 0 então
9          erro "underflow";
10     fim
11     x ← S[S.num_elem − 1];
12     Remove-Vetor(S, S.num_elem − 1);
13     retorna x;
14 fim
```

Na implementação por listas encadeadas, o topo da pilha é colocado no início da lista. Isso porque o procedimento `Remove-Lista` recebe como argumento o membro que antecede aquele que será removido, de forma a impossibilitar remoções eficientes.

```
1  função push(S, x)
2      Insere-Lista(S, S.head, x);
3  fim
4  função pop(S)
5      se S.head.proximo = NIL então
6          erro "underflow";
7      fim
8      x ← S.head.proximo;
9      Remove-Lista(S, S.head);
10     retorna ∗x;
11 fim
```

1.4 Filas

Uma fila, assim como a pilha, é uma estrutura de dados que implementa uma política para a ordem de inserções e remoções no conjunto. A política implementada para a fila é a **FIFO**: o primeiro elemento a entrar é também o primeiro a sair, assim como filas de atendimento. O ato de inserir numa fila é chamado de *enqueue*, enquanto o de remover de *dequeue*. Similarmente às pilhas, elas podem ser implementadas a partir de vetores ou listas encadeadas.

A forma mais intuitiva de criar uma fila com um vetor é inserir os novos membros

num extremo, e remover a partir do outro. Entretanto, supondo que as inserções são feitas no início e as remoções no fim do vetor, toda inserção implicaria na movimentação de todos os elementos já presentes. O mesmo valeria caso as remoções fossem no início.

Uma abordagem possível para realizar as operações de *enqueue* e *dequeue* em tempo constante é fazendo com que a fila seja circular, de forma a utilizar dois índices especiais: f e r, que indicam, respectivamente, o primeiro e o último elemento na fila. Quando a fila está inicialmente vazia, temos $f = 0$ e $r = n-1$, em que n é a capacidade da fila. Ao adicionar um novo membro, executamos $r \leftarrow (r+1) \bmod n$ e inserimos o elemento na posição r do vetor. A operação de *dequeue* é realizada retornando o membro apontado por f e depois fazendo $f \leftarrow (f+1) \bmod n$.

```
1  função enqueue(Q, x)
2      se Q.num_elem = Q.len então
3          erro "overflow";
4      fim
5      Q.r ← (Q.r + 1) mod Q.len;
6      Q[Q.r] ← *x;
7      Q.num_elem ← Q.num_elem + 1;
8  fim
9  função dequeue(Q)
10     se Q.num_elem = 0 então
11         erro "underflow";
12     fim
13     x ← Q[Q.f];
14     Q.f ← (Q.f + 1) mod Q.len;
15     Q.num_elem ← Q.num_elem − 1;
16     retorna x;
17 fim
```

A abordagem com listas encadeadas é substancialmente mais simples e intuitiva. A adição dum elemento novo é feita sempre ao final da própria lista, enquanto a remoção é feita no início. A razão é a mesma explicada para as pilhas: não é possível realizar a remoção do último elemento da lista sem buscar o elemento que o antecede, fazendo com que a operação não seja realizada em tempo constante.

```
1  função enqueue(Q, x)
2  |   Insere-Lista(Q, Q.tail, x);
3  fim
4  função dequeue(Q)
5  |   se Q.head.proximo = NIL então
6  |   |   erro "underflow";
7  |   fim
8  |   x ← *(Q.head.proximo);
9  |   Remove-Lista(Q, Q.head);
10 |   retorna x;
11 fim
```